Advanced Digital Logic

ENGR – UH 2310

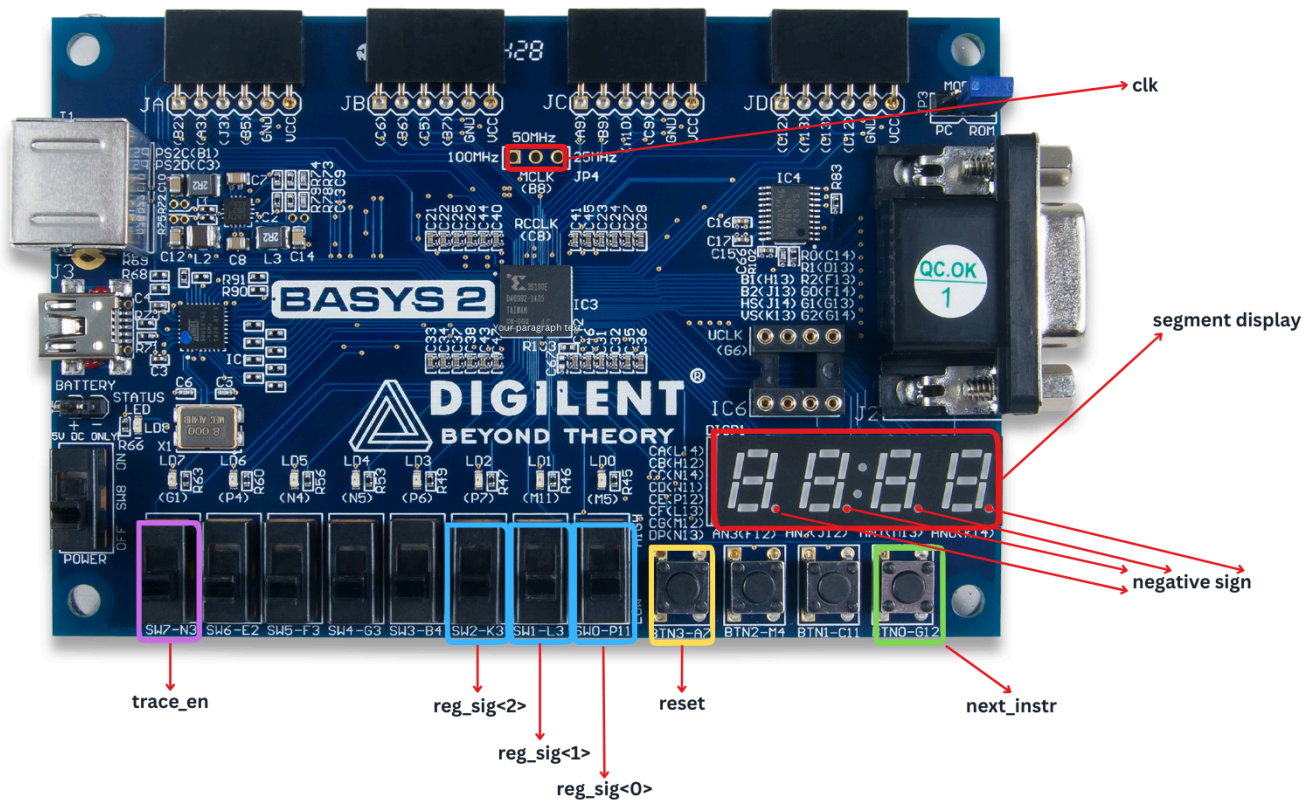
Spring II - 2025

Instructor: Muhammad Hassan Jamil

Group Members:

Name	NetID
Andre Llaneta	wpl5304
Hsin (Sheena) Chiang	hc3550
Laial Saad	ls6282
Zichen (Nancy) Jing	zj2256

Labelled FPGA



Code

Decoder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.common.all;

entity Decoder is
        port (  instruction_in : in STD_LOGIC_VECTOR (15 downto 0);
                opcode_out : out opcode_type;
                Rd_addr_out : out STD_LOGIC_VECTOR (2 downto 0);
                Rs1_addr_out : out STD_LOGIC_VECTOR (2 downto 0);
                Rs2_addr_out : out STD_LOGIC_VECTOR (2 downto 0);
                immediate_out : out STD_LOGIC_VECTOR (13 downto 0)
            );
end Decoder;

architecture Behavioral of Decoder is

signal opcode_internal : opcode_type;

-- intermediate signals for unpacking instruction fields
signal Rd     : STD_LOGIC_VECTOR(2 downto 0);
signal Rs1    : STD_LOGIC_VECTOR(2 downto 0);
signal Rs2    : STD_LOGIC_VECTOR(2 downto 0);
signal immediate_val          : STD_LOGIC_VECTOR(5 downto 0);

begin
        opcode_out <= opcode_internal;
        opcode_internal <= std_logic_vector_to_opcode_type( instruction_in(15 downto 12) );

-- unpack register numbers directly from instruction bits
        Rd_addr_out <= instruction_in(11 downto 9);
        Rs1_addr_out <= instruction_in(8 downto 6);
        Rs2_addr_out <= instruction_in(5 downto 3);
-- extract the 6-bit immediate from the instruction
        immediate_val <= instruction_in(5 downto 0);




        process(opcode_internal, Rs2, Rd, immediate_val)
begin
  if opcode_internal = OP_ANDI then
     immediate_out <= "11111111" & immediate_val; -- ANDI: upper bits are all 1s
```

```
    elsif (opcode_internal = OP_ORI) or (opcode_internal = OP_XORI) then
        immediate_out <= "00000000" & immediate_val; -- ORI, XORI are 0s

    elsif (opcode_internal = OP_ADDI) or (opcode_internal = OP_SUBI) then
        immediate_out <= (13 downto 6 => immediate_val(5)) & immediate_val; -- ADDI, SUBI
will make sign-extend by 5 bits

    elsif (opcode_internal = OP_SLL) or (opcode_internal = OP_SRL) then
        immediate_out <= (10 downto 0 => '0') & immediate_val(2 downto 0); -- SLL, SRL only
lower 3 bits are used for shift

    elsif (opcode_internal = OP_BLT) or
          (opcode_internal = OP_BE)  or
          (opcode_internal = OP_JMP) then
        immediate_out <= (13 downto 6 => immediate_val(5)) & immediate_val; -- BLT, BE, JMP
sign-extend bit 5 for branch or jump

    else

        immediate_out <= (others => '0'); -- default thus no immediate
    end if;
end process;

end Behavioral;
```

Controller.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.common.all;

entity Controller is
	port (	opcode : in opcode_type;

		operand_1 : out STD_LOGIC_VECTOR (13 downto 0);
		operand_2 : out STD_LOGIC_VECTOR (13 downto 0);
		result : in STD_LOGIC_VECTOR (13 downto 0);
		curr_PC : in STD_LOGIC_VECTOR (6 downto 0);
		new_PC : out STD_LOGIC_VECTOR (6 downto 0);
		PC_we : out STD_LOGIC;
		PC_incr : out STD_LOGIC;
		Rs1_data : in STD_LOGIC_VECTOR (13 downto 0);
		Rs2_data : in STD_LOGIC_VECTOR (13 downto 0);
		immediate : in STD_LOGIC_VECTOR (13 downto 0);
		Rd_we : out STD_LOGIC;
		Rd_data : out STD_LOGIC_VECTOR (13 downto 0)
	);
end Controller;

architecture Behavioral of Controller is


begin
control : process (opcode, Rs1_data, Rs2_data, result, immediate, curr_PC)
begin
	-- default assignments, can be overwritten below
	operand_1 <= Rs1_data;
	operand_2 <= Rs2_data;
	Rd_we <= '0';
	Rd_data <= result;
	PC_we <= '0';
	new_PC <= (others => 'X');
	PC_incr <= '0';

	-- regular operations with Rs1, Rs2, Rd
	if (opcode = OP_ADD or opcode = OP_SUB or opcode = OP_AND or opcode =
OP_OR or
		opcode = OP_XOR) then

		Rd_we <= '1';
		PC_incr <= '1';
```

```
        -- immediate operations
        elsif (opcode = OP_ADDI or opcode = OP_SUBI or opcode = OP_ANDI or
              opcode = OP_ORI or opcode = OP_XORI or opcode = OP_SLL or opcode =
OP_SRL) then

                operand_2 <= immediate;
                Rd_we <= '1';
                PC_incr <= '1';

        -- branch operations
        elsif (opcode = OP_BE) then
                if Rs1_data = Rs2_data then
                        operand_1 <=
                        STD_LOGIC_VECTOR(to_unsigned(to_integer(unsigned(curr_PC)),
14));
                        operand_2 <= immediate;
                        PC_we <= '1';
                        new_PC <= result (6 downto 0);
                else
                        PC_incr <= '1';
                end if;

        elsif (opcode = OP_BLT) then
                if signed(Rs1_data) < signed(Rs2_data) then
                        operand_1 <=
STD_LOGIC_VECTOR(to_unsigned(to_integer(unsigned(curr_PC)), 14));
                        operand_2 <= immediate;
                        PC_we <= '1';
                        new_PC <= result (6 downto 0);
                else
                        PC_incr <= '1';
                end if;

        elsif (opcode = OP_JMP) then
                operand_1 <=
STD_LOGIC_VECTOR(to_unsigned(to_integer(unsigned(curr_PC)), 14));
                operand_2 <= immediate;
                PC_we <= '1';
                new_PC <= result (6 downto 0);

        -- only OP_HALT remains

        else
                operand_1 <= (13 downto 0 => 'X');
                operand_2 <= (13 downto 0 => 'X');
                Rd_data <= (13 downto 0 => 'X');
        end if;
end process;

end Behavioral;
```

ALU.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.common.all;

entity ALU is
	port (	operand_1 : in STD_LOGIC_VECTOR (13 downto 0);
		operand_2 : in STD_LOGIC_VECTOR (13 downto 0);

		opcode : in opcode_type;

		result : out STD_LOGIC_VECTOR (13 downto 0);
		overflow : out STD_LOGIC

	);
end ALU;

architecture Behavioral of ALU is

signal result_internal: STD_LOGIC_VECTOR (13 downto 0);

begin

result <= result_internal;

calculate : process (operand_1, operand_2, opcode)
begin
	-- Signed addition for  OP_ADD, OP_ADDI, OP_BE, OP_BLT, and OP_JMP
	if ((opcode = OP_ADD) or (opcode = OP_ADDI) or (opcode = OP_BE) or (opcode =
OP_BLT) or (opcode = OP_JMP)) then

		result_internal <= std_logic_vector( signed(operand_1) + signed(operand_2) );

	-- Signed subtraction for both OP_SUB and OP_SUBI
	elsif ((opcode = OP_SUB) or (opcode = OP_SUBI)) then
		result_internal <= std_logic_vector(signed(operand_1) - signed(operand_2));

	-- Bitwise AND for both register and immediate versions
	elsif ((opcode = OP_AND) or (opcode = OP_ANDI)) then
		result_internal <= operand_1 and operand_2;

	-- Bitwise OR for both register and immediate versions
	elsif ((opcode = OP_OR) or (opcode = OP_ORI)) then
		result_internal <= operand_1 or operand_2;

	-- Bitwise XOR for both register and immediate versions
```

```
        elsif ((opcode = OP_XOR) or (opcode = OP_XORI)) then
                result_internal <= operand_1 xor operand_2;

        -- Shift Logical Left
        elsif opcode = OP_SLL then
                result_internal <= std_logic_vector( shift_left(unsigned(operand_1),
to_integer(unsigned(operand_2))) );

        -- Shift Logical Right
        elsif opcode = OP_SRL then
                result_internal <= std_logic_vector(shift_right(unsigned(operand_1),
to_integer(unsigned(operand_2))));

        -- For HALT, output undefined value
        elsif opcode = OP_HALT then
                result_internal <= (others => '0');


        -- Default: assign undefined value
        else
                result_internal <= (others => '0');
end if;
end process;

-- Detection of overflow for all signed arithmetic operations
ofl : process (operand_1, operand_2, result_internal, opcode)
begin
overflow <= '0'; -- Default case: No overflow
  if (opcode = OP_ADD or opcode = OP_ADDI) then
                if ((operand_1(13) = operand_2(13)) and (result_internal(13) /=
operand_1(13))) then  -- For addition (OP_ADD, OP_ADDI), overflow occurs if both operands
have the same sign


                                                -- but the result has a different sign (indicating
overflow).
                        overflow <= '1';
                end if;


  elsif (opcode = OP_SUB or opcode = OP_SUBI) then
                if ((operand_1(13) /= operand_2(13)) and (result_internal(13) /=
operand_1(13))) then -- For subtraction (OP_SUB, OP_SUBI), overflow occurs if the
operands have different signs


                                                -- but the result has a different sign than
operand_1 (indicating overflow).
                        overflow <= '1';
                end if;
```

```
  end if;
end process ofl;

end Behavioral;
```

Top_processor_FGPA.vhd

```
-- Reformatted top_processor_FPGA.vhd without trace logic
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.common.all;

entity top_processor_FPGA is
    port (
        next_instr : in  STD_LOGIC;
        clk        : in  STD_LOGIC;
        rst        : in  STD_LOGIC;
        -- Display interface
        seg_bits   : out STD_LOGIC_VECTOR(0 to 7);
        seg_an     : out STD_LOGIC_VECTOR(3 downto 0)
    );
end entity top_processor_FPGA;

architecture Behavioral of top_processor_FPGA is

    -- Component declarations
    component Display_Controller
        port (
            clk       : in  STD_LOGIC;
            rst       : in  STD_LOGIC;
            opcode    : in  opcode_type;
            operand_1 : in  STD_LOGIC_VECTOR(13 downto 0);
            operand_2 : in  STD_LOGIC_VECTOR(13 downto 0);
            result    : in  STD_LOGIC_VECTOR(13 downto 0);
            overflow  : in  STD_LOGIC;
            seg_bits  : out STD_LOGIC_VECTOR(0 to 7);
            seg_an    : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component Display_Controller;

    component Instructions_ROM
        port (
            address_in : in  STD_LOGIC_VECTOR(6 downto 0);
            data_out   : out STD_LOGIC_VECTOR(15 downto 0)
        );
    end component Instructions_ROM;

    component Decoder
        port (
            instruction_in : in  STD_LOGIC_VECTOR(15 downto 0);
            opcode_out     : out opcode_type;
            Rd_addr_out    : out STD_LOGIC_VECTOR(2 downto 0);
            Rs1_addr_out   : out STD_LOGIC_VECTOR(2 downto 0);
```

```
        Rs2_addr_out   : out STD_LOGIC_VECTOR(2 downto 0);
        immediate_out  : out STD_LOGIC_VECTOR(13 downto 0)
    );
end component Decoder;

component Controller
    port (
        opcode        : in  opcode_type;
        operand_1     : out STD_LOGIC_VECTOR(13 downto 0);
        operand_2     : out STD_LOGIC_VECTOR(13 downto 0);
        result        : in  STD_LOGIC_VECTOR(13 downto 0);
        curr_PC       : in  STD_LOGIC_VECTOR(6 downto 0);
        new_PC        : out STD_LOGIC_VECTOR(6 downto 0);
        PC_we         : out STD_LOGIC;
        PC_incr       : out STD_LOGIC;
        Rs1_data      : in  STD_LOGIC_VECTOR(13 downto 0);
        Rs2_data      : in  STD_LOGIC_VECTOR(13 downto 0);
        immediate     : in  STD_LOGIC_VECTOR(13 downto 0);
        Rd_we         : out STD_LOGIC;
        Rd_data       : out STD_LOGIC_VECTOR(13 downto 0)
    );
end component Controller;

component PC
    port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        PC_in   : in  STD_LOGIC_VECTOR(6 downto 0);
        PC_out  : out STD_LOGIC_VECTOR(6 downto 0);
        PC_we   : in  STD_LOGIC;
        PC_incr : in  STD_LOGIC
    );
end component PC;

component Registers
    port (
        clk           : in  STD_LOGIC;
        rst           : in  STD_LOGIC;
        Rs1_addr_in   : in  STD_LOGIC_VECTOR(2 downto 0);
        Rs1_data_out  : out STD_LOGIC_VECTOR(13 downto 0);
        Rs2_addr_in   : in  STD_LOGIC_VECTOR(2 downto 0);
        Rs2_data_out  : out STD_LOGIC_VECTOR(13 downto 0);
        Rd_addr_in    : in  STD_LOGIC_VECTOR(2 downto 0);
        Rd_data_in    : in  STD_LOGIC_VECTOR(13 downto 0);
        Rd_we         : in  STD_LOGIC
    );
end component Registers;

component ALU
    port (
```

```
        operand_1 : in  STD_LOGIC_VECTOR(13 downto 0);
        operand_2 : in  STD_LOGIC_VECTOR(13 downto 0);
        opcode    : in  opcode_type;
        result    : out STD_LOGIC_VECTOR(13 downto 0);
        overflow  : out STD_LOGIC
    );
end component ALU;

-- Internal signals
signal clk_proc          : STD_LOGIC;
signal rst_disp          : STD_LOGIC;
signal curr_PC_sig       : STD_LOGIC_VECTOR(6 downto 0);
signal instruction_sig   : STD_LOGIC_VECTOR(15 downto 0);
signal new_PC_sig        : STD_LOGIC_VECTOR(6 downto 0);
signal PC_we_sig         : STD_LOGIC;
signal PC_incr_sig       : STD_LOGIC;
signal opcode_internal   : opcode_type;
signal Rd_addr_sig       : STD_LOGIC_VECTOR(2 downto 0);
signal Rs1_addr_sig      : STD_LOGIC_VECTOR(2 downto 0);
signal Rs2_addr_sig      : STD_LOGIC_VECTOR(2 downto 0);
signal immediate_sig     : STD_LOGIC_VECTOR(13 downto 0);
signal Rd_data_sig       : STD_LOGIC_VECTOR(13 downto 0);
signal Rs1_data_sig      : STD_LOGIC_VECTOR(13 downto 0);
signal Rs2_data_sig      : STD_LOGIC_VECTOR(13 downto 0);
signal Rd_we_sig         : STD_LOGIC;
signal operand_1_internal : STD_LOGIC_VECTOR(13 downto 0);
signal operand_2_internal : STD_LOGIC_VECTOR(13 downto 0);
signal result_internal   : STD_LOGIC_VECTOR(13 downto 0);
signal overflow_internal  : STD_LOGIC;

begin

-- Simplified clock and reset
clk_proc <= next_instr;
rst_disp <= rst or clk_proc;

-- Component instantiations
Display_Controller_inst : Display_Controller
    port map (
        clk       => clk,
        rst       => rst_disp,
        opcode    => opcode_internal,
        operand_1 => operand_1_internal,
        operand_2 => operand_2_internal,
        result    => result_internal,
        overflow  => overflow_internal,
        seg_bits  => seg_bits,
        seg_an    => seg_an
    );
```

```
Instructions_ROM_inst : Instructions_ROM
   port map (
      address_in => curr_PC_sig,
      data_out   => instruction_sig
   );

Decoder_inst : Decoder
   port map (
      instruction_in => instruction_sig,
      opcode_out     => opcode_internal,
      Rd_addr_out    => Rd_addr_sig,
      Rs1_addr_out   => Rs1_addr_sig,
      Rs2_addr_out   => Rs2_addr_sig,
      immediate_out  => immediate_sig
   );

Controller_inst : Controller
   port map (
      opcode    => opcode_internal,
      operand_1 => operand_1_internal,
      operand_2 => operand_2_internal,
      result    => result_internal,
      curr_PC   => curr_PC_sig,
      new_PC    => new_PC_sig,
      PC_we     => PC_we_sig,
      PC_incr   => PC_incr_sig,
      Rs1_data  => Rs1_data_sig,
      Rs2_data  => Rs2_data_sig,
      immediate => immediate_sig,
      Rd_we     => Rd_we_sig,
      Rd_data   => Rd_data_sig
   );

PC_inst : PC
   port map (
      clk     => clk_proc,
      rst     => rst,
      PC_in   => new_PC_sig,
      PC_out  => curr_PC_sig,
      PC_we   => PC_we_sig,
      PC_incr => PC_incr_sig
   );

Registers_inst : Registers
   port map (
      clk           => clk_proc,
      rst           => rst,
      Rs1_addr_in   => Rs1_addr_sig,
      Rs1_data_out  => Rs1_data_sig,
      Rs2_addr_in   => Rs2_addr_sig,
```

```
        Rs2_data_out  => Rs2_data_sig,
        Rd_addr_in    => Rd_addr_sig,
        Rd_data_in    => Rd_data_sig,
        Rd_we         => Rd_we_sig
    );

  ALU_inst : ALU
    port map (
        operand_1 => operand_1_internal,
        operand_2 => operand_2_internal,
        opcode    => opcode_internal,
        result    => result_internal,
        overflow  => overflow_internal
    );

end architecture Behavioral;
```

Instructions_absMAX.vhd

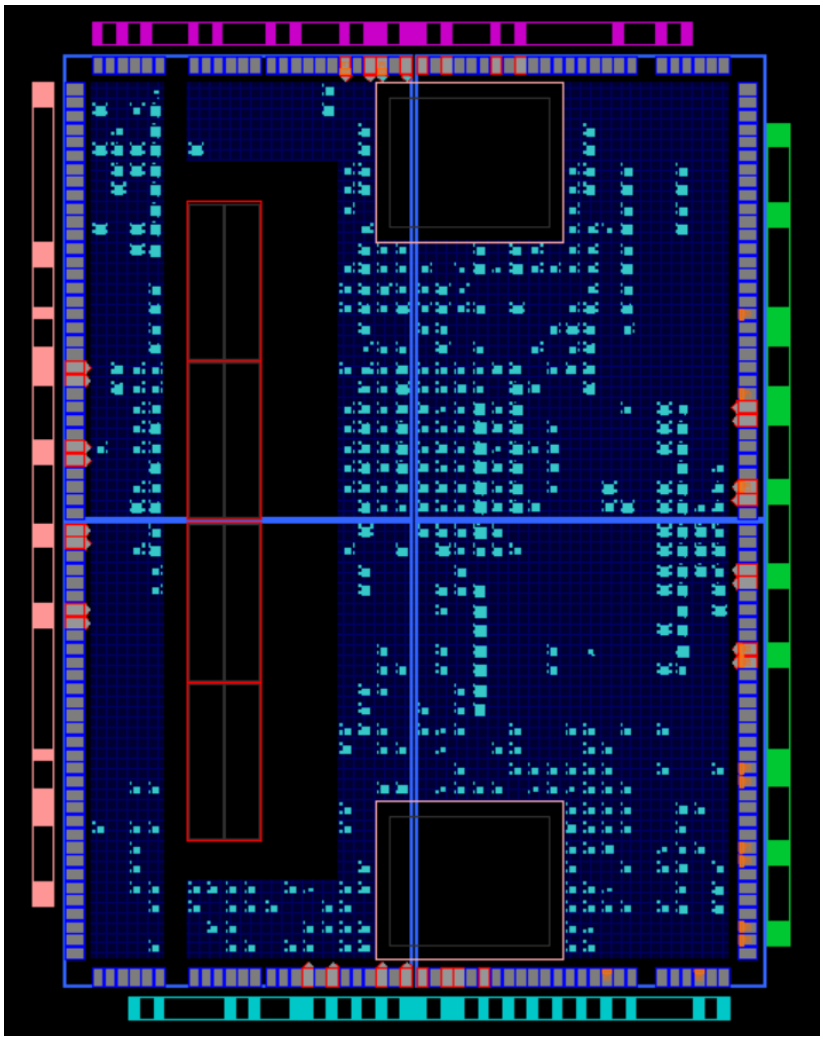
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.common.all;

entity Instructions_ROM is
	port (	address_in : in STD_LOGIC_VECTOR (6 downto 0);
		data_out : out STD_LOGIC_VECTOR (15 downto 0)
	      );
end Instructions_ROM;

architecture Behavioral of Instructions_ROM is

type ROM_type is array (0 to 127) of STD_LOGIC_VECTOR (15 downto 0);
signal rom : ROM_type;

begin
	data_out <= rom ( to_integer(unsigned(address_in)) );

	rom_process : process (address_in)
	begin
		-- LEAVE AS IS
		--
		-- reset ROM content completely with HLT operations; note that loop will be
unrolled during synthesis
		for i in 0 to 127 loop
			rom(i) <= opcode_type_to_std_logic_vector(OP_HALT) & (11 downto 0
=> 'X');
		end loop;

		-- Initialization (inputs into R1 and R2)
		rom(0)  <= opcode_type_to_std_logic_vector(OP_ADDI) & "001" & "000" &
"000101"; -- R1 = 5
		rom(1)  <= opcode_type_to_std_logic_vector(OP_ADDI) & "010" & "000" &
"001011"; -- R2 = 11

		-- Compute R3 = R1 - R2
		rom(2)  <= opcode_type_to_std_logic_vector(OP_SUB) & "011" & "001" &
"010" & "000"; -- R3 = R1 - R2

		-- If R3 < 0, negate R3: R3 = -R3
		rom(3)  <= opcode_type_to_std_logic_vector(OP_BLT) & "000" & "011" & "000"
& "010"; -- if R3 < 0, jump +2 to negate
		rom(4)  <= opcode_type_to_std_logic_vector(OP_JMP) & "000" & "000" &
"000" & "010"; -- else skip negate
		rom(5)  <= opcode_type_to_std_logic_vector(OP_SUB) & "011" & "000" &
```

```
"011" & "000"; -- R3 = 0 - R3 (negate)


                    -- Show absolute difference
                    rom(6) <= opcode_type_to_std_logic_vector(OP_ADDI) & "011" & "011" &
"000" & "000"; -- show R3

                    -- Done
                    rom(7) <= opcode_type_to_std_logic_vector(OP_HALT) & (11 downto 0 => 'X');

                    -- LEAVE AS IS
                    --
                    -- This dummy code below helps to infer all 7 registers (R1--R7) in the register
file, without short-cuts into combinatorial
                    -- logic due to tool optimization; that is important to ensure the same baseline
hardware configuration for all
                    rom(121) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"001_001_000_000";
                    rom(122) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"010_010_001_000";
                    rom(123) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"011_011_010_000";
                    rom(124) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"100_100_011_000";
                    rom(125) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"101_101_100_000";
                    rom(126) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"110_110_101_000";
                    rom(127) <= opcode_type_to_std_logic_vector(OP_ADD) &
b"111_111_110_000";

          end process;

end Behavioral;
```

Task 1. Implement the microprocessor in VHDL

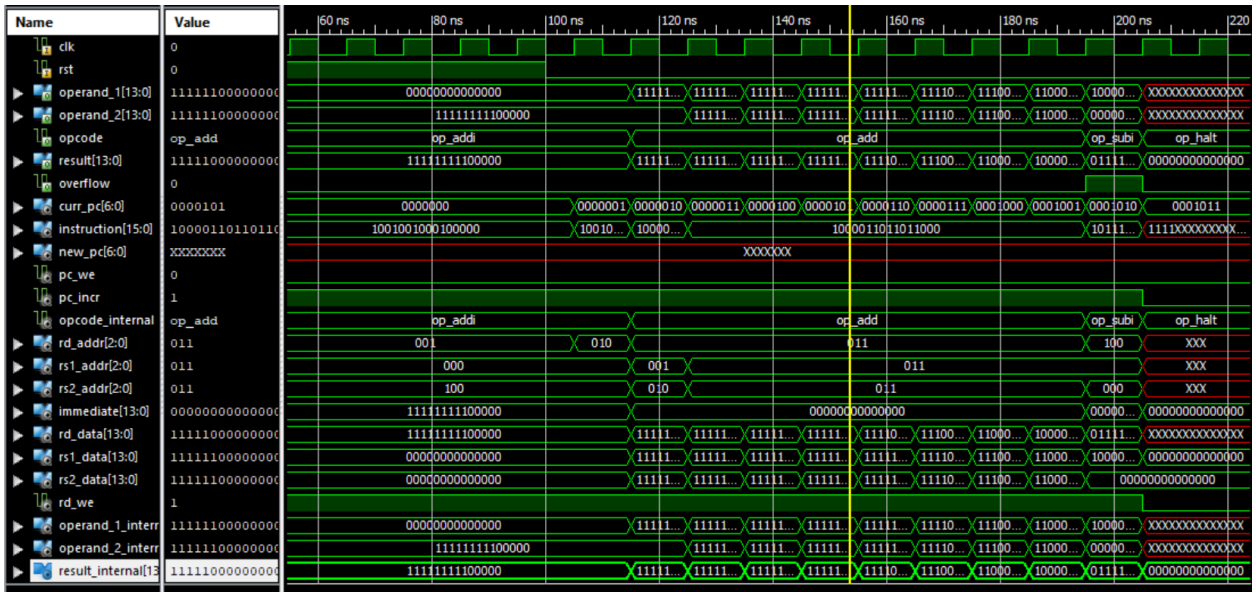
It takes one clock cycle to complete one instruction. Which includes reading the instruction and executing the result.

Floorplan Snapshot



Task 2. Test the microprocessor

Snapshots for the Behavioural Simulation



Task 3. Evaluate binary program

Describe the Program Behaviour

1. **1001 001 000 000 101**

Opcode: 1001 (ADDI - Add Immediate)
Destination register (Rd): 001 (R1)
Source register (Rs1): 000 (R0)
Immediate: 000 101 (5 in decimal)
Operation: since R0 is assumed to be 0, R1=R0+5=0+5=5
After execution:
R1 = 5

2. **1001 010 000 000 000**

Opcode: 1001 (ADDI - Add Immediate)
Destination register (Rd): 010 (R2)
Source register (Rs1): 000 (R0)
Immediate: 000 000 (0 in decimal)
Operation: R2=R0+0=0+0=0
After execution:
R2 = 0

3. **1001 010 010 000 101**

Opcode: 1001 (ADDI - Add Immediate)
Destination register (Rd): 010 (R2)
Source register (Rs1): 010 (R2, which is currently 0 from second execution)
Immediate: 000 101 (5 in decimal)
Operation: R2=R2+5=0+5=5
After execution:
R2 = 5

4. **1011 001 001 000 001**

Opcode: 1011 (SUBI - Subtract Immediate)
Destination register (Rd): 001 (R1)
Source register (Rs1): 001 (R1, which is currently 5 from first execution)
Immediate: 000 001 (1 in decimal)
Operation: R1=R1−1=5−1=4
After execution:
R1 = 4

5. **1100 111 000 001 110**

Opcode: 1100 (BLT - Branch If Less Than)
Source register (Rs1): 000 (R0, which was assumed 0)

Source register (Rs2): 001 (R1, which is currently 4 from the fourth instruction)
Immediate: 001 110 (14 in decimal)
Operation: Since Rs1 < Rs2 (0 < 4), the program counter (PC) is advanced by the immediate value 14, so execution jumps to instruction at address 18.

Final Register State:
R0 = 0 (constant)
R1 = 4
R2 = 5
R0 = R7 = 0

Task 4. Own program for the microprocessor

Program Description

We wrote a program that computes the absolute difference between two signed inputs (stored in R1 and R2). To compute this, we first subtract R2 from R1. This result is stored in R3, which is compared with 0 to determine whether the result is negative. Then, using a BLT instruction, it decides whether to negate the result to get the absolute value. If it is negative, the program jumps to an operation that subtracts R3 from 0; otherwise, R3 remains the same. The program ends by displaying the result via dummy ADD instructions. This implementation works for any pair of signed input values within given bits and always results in a positive difference.

Expected Behaviour

If we preload:
R1 = 11
R2 = 5
→ The program will simply display the difference of 6.

If instead, we do as shown in our demonstration where we preload:
R1 = 5
R2 = 11
→ the program computes R3 = -6, then negates it and loads it into R3, resulting in R3 = 6.

→ The difference will be a positive number no matter what.

Contribution
Andre: Task 1,2,3 Code
Laial: Task 1,4 Code, Lab Report
Nancy: Task 4 Code, Lab Report
Sheena: Task 1,3,4 Code